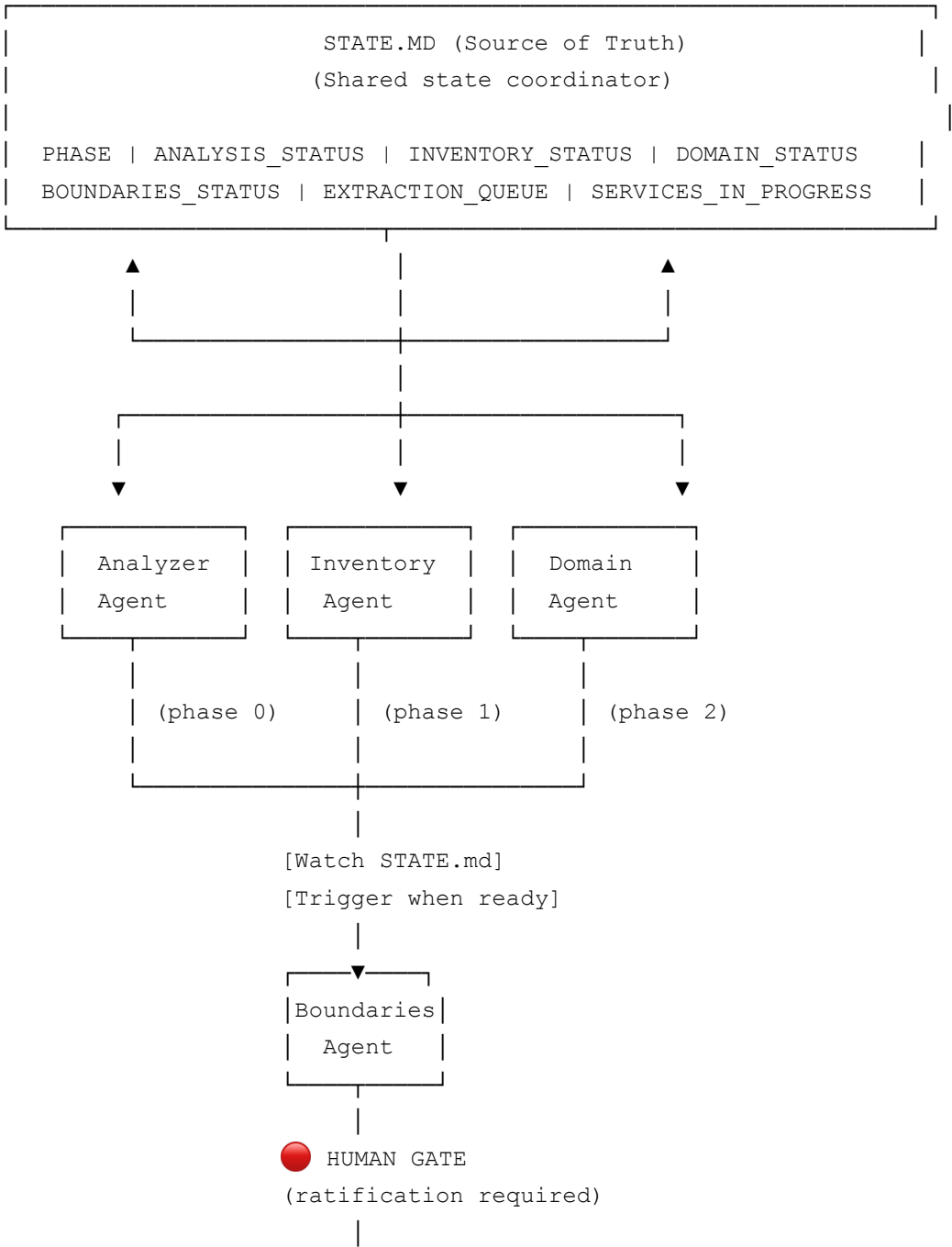
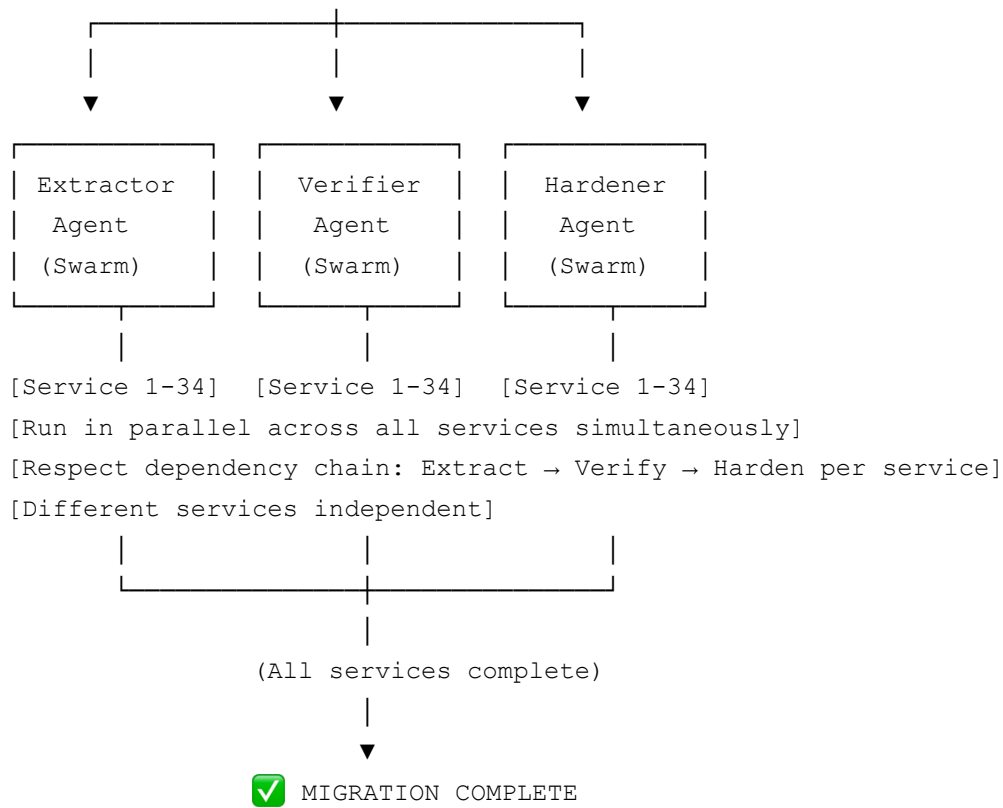


Agentic Swarm Architecture for Monolith Migration

Paradigm shift: From sequential orchestration → parallel autonomous swarm

Architecture Overview





Key Principle: Event-Driven Coordination

Agents don't wait for commands. They:

1. Watch STATE.md for changes
 2. Check if prerequisites for their phase are met
 3. Run autonomously when ready
 4. Update STATE.md with results
 5. Wake up other agents that depend on them
-

STATE.md Structure (Swarm-Optimized)

```
# GLOBAL STATE
PHASE: "extraction-loop"
HUMAN_GATE_PENDING: false
SWARM_ACTIVE: true
LAST_UPDATE: "2026-06-09T14:32:00Z"
```

```
# PHASE 0: ANALYSIS
PHASE_0:
  STATUS: "complete"
  STARTED: "2026-06-09T10:00:00Z"
  COMPLETED: "2026-06-09T10:10:00Z"
  FILES_GENERATED: 9
  AGENT_RUNNING: false

# PHASE 1: INVENTORY
PHASE_1:
  STATUS: "complete"
  STARTED: "2026-06-09T10:10:00Z"
  COMPLETED: "2026-06-09T10:25:00Z"
  MODULES_FOUND: 34
  AGENT_RUNNING: false

# PHASE 2: DOMAIN
PHASE_2:
  STATUS: "complete"
  STARTED: "2026-06-09T10:25:00Z"
  COMPLETED: "2026-06-09T10:45:00Z"
  DOMAINS_IDENTIFIED: 12
  AGENT_RUNNING: false

# PHASE 🚫: BOUNDARIES (Human Gate)
PHASE_3_GATE:
  STATUS: "ratified"
  RATIFIED_BY: "vivek@wells-fargo.com"
  RATIFIED_AT: "2026-06-09T11:30:00Z"
  RATIFIED_BOUNDARIES:
    - GetLoanAccountSupport
    - AccountDetail
    - PaymentProcessing
    # ... 9 more
  EXTRACTION_ORDER:
    [1] GetLoanAccountSupport
    [2] AccountDetail
    [3] PaymentProcessing
    # ... 31 more (ordered by risk)

# PHASE 3-5: EXTRACTION LOOP (Parallel Swarm)
EXTRACTION_QUEUE:
  TOTAL_SERVICES: 34
  COMPLETED: 3
  IN_PROGRESS: 4
  PENDING: 27
```

SERVICES:

GetLoanAccountSupport:

PRIORITY: 1
STATUS: "completed"
PHASE_3_STATUS: "complete"
STARTED: "2026-06-09T12:00:00Z"
COMPLETED: "2026-06-09T12:10:00Z"
EXTRACTOR_RUNNING: false
PHASE_4_STATUS: "complete"
STARTED: "2026-06-09T12:10:00Z"
COMPLETED: "2026-06-09T12:25:00Z"
VERIFIER_RUNNING: false
VERDICT: "PASS"
PHASE_5_STATUS: "complete"
STARTED: "2026-06-09T12:25:00Z"
COMPLETED: "2026-06-09T12:40:00Z"
HARDENER_RUNNING: false
CUTOVER_STATUS: "soak-complete"
MANUAL_DEPLOYMENT: "2026-06-09T16:00:00Z"
CANARY_1_COMPLETE: "2026-06-09T16:15:00Z"
CANARY_2_COMPLETE: "2026-06-09T16:30:00Z"
CANARY_3_COMPLETE: "2026-06-09T17:00:00Z"
PRODUCTION_100_PERCENT: "2026-06-09T17:00:00Z"
SOAK_COMPLETE: "2026-06-10T17:00:00Z"
DECOMMISSION_COMPLETE: "2026-06-10T18:00:00Z"

AccountDetail:

PRIORITY: 2
STATUS: "in-progress"
PHASE_3_STATUS: "in-progress"
STARTED: "2026-06-09T12:05:00Z"
EXTRACTOR_RUNNING: true
EXTRACTOR_PID: "agent-ex-2"
PHASE_4_STATUS: "pending"
PHASE_5_STATUS: "pending"

PaymentProcessing:

PRIORITY: 3
STATUS: "in-progress"
PHASE_3_STATUS: "complete"
STARTED: "2026-06-09T12:10:00Z"
COMPLETED: "2026-06-09T12:20:00Z"
PHASE_4_STATUS: "in-progress"
STARTED: "2026-06-09T12:20:00Z"
VERIFIER_RUNNING: true
VERIFIER_PID: "agent-vf-1"
PHASE_5_STATUS: "pending"

```
OnlineWires:
  PRIORITY: 4
  STATUS: "pending"
  PHASE_3_STATUS: "pending"
  PHASE_4_STATUS: "pending"
  PHASE_5_STATUS: "pending"

# ... 30 more services
```

Agent Behaviors in Swarm

Analyzer Agent (Runs once, parallel with nothing)

```
LOOP every 5 seconds:
  IF STATE.PHASE == "start" AND STATE.PHASE_0.STATUS != "complete":
    AND STATE.PHASE_0.AGENT_RUNNING == false:

    SET: STATE.PHASE_0.AGENT_RUNNING = true
    RUN: business-logic-analyzer skill
    UPDATE: STATE.PHASE_0.STATUS = "complete"
    SET: STATE.PHASE_0.AGENT_RUNNING = false

    SIGNAL: Inventory & Domain agents to wake up
```

Inventory Agent (Runs once, parallel with nothing)

```
LOOP every 5 seconds:
  IF STATE.PHASE_1.STATUS != "complete"
    AND STATE.PHASE_0.STATUS == "complete"
    AND STATE.PHASE_1.AGENT_RUNNING == false:

    SET: STATE.PHASE_1.AGENT_RUNNING = true
    RUN: migration-inventory agent
    UPDATE: STATE.PHASE_1.STATUS = "complete"
    SET: STATE.PHASE_1.AGENT_RUNNING = false

    SIGNAL: Domain & Boundaries agents to wake up
```

Domain Agent (Runs once, parallel with Inventory)

```

LOOP every 5 seconds:
  IF STATE.PHASE_2.STATUS != "complete"
    AND STATE.PHASE_0.STATUS == "complete"
    AND STATE.PHASE_2.AGENT_RUNNING == false:

    SET: STATE.PHASE_2.AGENT_RUNNING = true
    RUN: migration-domain agent
    UPDATE: STATE.PHASE_2.STATUS = "complete"
    SET: STATE.PHASE_2.AGENT_RUNNING = false

    SIGNAL: Boundaries agent to wake up

```

Boundaries Agent (Runs once, pauses at human gate)

```

LOOP every 5 seconds:
  IF STATE.PHASE_3_GATE.STATUS != "ratified"
    AND STATE.PHASE_1.STATUS == "complete"
    AND STATE.PHASE_2.STATUS == "complete":

    RUN: migration-boundaries agent
    DISPLAY: Boundary ratification UI
    SET: STATE.HUMAN_GATE_PENDING = true

    WAIT: until STATE.PHASE_3_GATE.STATUS == "ratified"
    SET: STATE.HUMAN_GATE_PENDING = false

    SIGNAL: Extractor agents to wake up

```

Extractor Agent (Swarm - Multiple Concurrent Instances)

```

LOOP every 2 seconds:
  IF STATE.EXTRACTION_QUEUE exists
    AND STATE.PHASE_3_GATE.STATUS == "ratified":

    FIND: Next service in EXTRACTION_ORDER
      WHERE STATE.SERVICES[service].PHASE_3_STATUS == "pending"
      AND NOT already running

    IF service found AND parallel slots available:
      SET: STATE.SERVICES[service].PHASE_3_STATUS = "in-progress"
      SET: STATE.SERVICES[service].PHASE_3_STARTED = now()
      SET: STATE.SERVICES[service].EXTRACTOR_RUNNING = true

      SPAWN: migration-extractor agent for this service

```

WAIT: for completion

UPDATE: STATE.SERVICES[service].PHASE_3_STATUS = "complete"

SET: STATE.SERVICES[service].EXTRACTOR_RUNNING = false

SIGNAL: Verifier agent to wake up for this service

Verifier Agent (Swarm - Multiple Concurrent Instances)

LOOP every 2 seconds:

IF STATE.EXTRACTION_QUEUE exists:

FIND: Service where:

STATE.SERVICES[service].PHASE_3_STATUS == "complete"

AND STATE.SERVICES[service].PHASE_4_STATUS != "complete"

AND STATE.SERVICES[service].VERIFIER_RUNNING == false

IF service found AND parallel slots available:

SET: STATE.SERVICES[service].PHASE_4_STATUS = "in-progress"

SET: STATE.SERVICES[service].PHASE_4_STARTED = now()

SET: STATE.SERVICES[service].VERIFIER_RUNNING = true

SPAWN: migration-verifier agent for this service

WAIT: for completion

IF VERDICT == "PASS":

UPDATE: STATE.SERVICES[service].PHASE_4_STATUS = "complete"

SET: STATE.SERVICES[service].VERIFIER_RUNNING = false

SIGNAL: Hardener agent to wake up

IF VERDICT == "FAIL":

UPDATE: STATE.SERVICES[service].PHASE_4_STATUS = "failed"

SET: STATE.SERVICES[service].VERIFIER_RUNNING = false

FLAG: Notify user via email (manual retry needed)

Hardener Agent (Swarm - Multiple Concurrent Instances)

LOOP every 2 seconds:

IF STATE.EXTRACTION_QUEUE exists:

FIND: Service where:

STATE.SERVICES[service].PHASE_4_STATUS == "complete"

```
AND STATE.SERVICES[service].PHASE_5_STATUS != "complete"
AND STATE.SERVICES[service].HARDENER_RUNNING == false

IF service found AND parallel slots available:
  SET: STATE.SERVICES[service].PHASE_5_STATUS = "in-progress"
  SET: STATE.SERVICES[service].PHASE_5_STARTED = now()
  SET: STATE.SERVICES[service].HARDENER_RUNNING = true

  SPAWN: migration-hardener agent for this service

  WAIT: for completion

  UPDATE: STATE.SERVICES[service].PHASE_5_STATUS = "complete"
  SET: STATE.SERVICES[service].HARDENER_RUNNING = false

  SIGNAL: User that service is ready for manual cutover
  EMAIL: "Service ${service} ready for production cutover"

  SET: STATE.SERVICES[service].AWAITING_MANUAL_CUTOVER = true
```

Concurrency Control

Parallel Limits

```
SWARM_CONFIG:
  MAX_PARALLEL_EXTRACTORS: 4
    # Run 4 extractions simultaneously
    # (Limited by monolith repo lock contention)

  MAX_PARALLEL_VERIFIERS: 6
    # Run 6 verifications simultaneously
    # (Limited by test environment capacity)

  MAX_PARALLEL_HARDENERS: 6
    # Run 6 hardening tasks simultaneously
    # (No I/O contention, CPU bound)

  EXTRACTION_TIMEOUT: 600 seconds (10 mins)
  VERIFICATION_TIMEOUT: 900 seconds (15 mins)
  HARDENING_TIMEOUT: 900 seconds (15 mins)

  RETRY_ON_TIMEOUT: 2
    # If agent exceeds timeout, retry up to 2x
```


Dependency Chain (Per Service)

Service 1: [Extract] → [Verify] → [Harden] → [Cutover]
Service 2: [Extract] → [Verify] → [Harden] → [Cutover]
Service 3: [Extract] → [Verify] → [Harden] → [Cutover]
...
Service 34: [Extract] → [Verify] → [Harden] → [Cutover]

Constraint: Each service must do Extract BEFORE Verify BEFORE Harden
But: Different services are completely independent
Result: Full parallelism across 34 services

Timeline: Sequential vs Swarm

Sequential (Current)

Analysis:	10 min	(1 instance)
Inventory:	15 min	(1 instance)
Domain:	15 min	(1 instance)
Boundaries:	2 hours	(human decision)

Loop 34 services:

Service 1:	40 min (extract + verify + harden)
Service 2:	40 min
...	
Service 34:	40 min

+ Manual cutover: 27 hours per service (soak)

Total: 1,360 hours = 56 days

Swarm Parallel (New)

Analysis:	10 min	(1 instance)
Inventory:	15 min	(1 instance, can start after analysis)
Domain:	15 min	(1 instance, can start after analysis)
Boundaries:	2 hours	(human decision)

Extraction Phase:

4 services in parallel:

Service 1-4: 40 min each

Total time: 40 min (not 160 min)

Verification Phase:

6 services in parallel:

Service 1-6: 40 min each

Total time: 40 min (not 240 min)

Hardening Phase:

6 services in parallel:

Service 1-6: 40 min each

Total time: 40 min (not 240 min)

Loop continues for all 34 services

Total extraction+verify+harden: ~4 hours (not 1,300 hours)

+ Manual cutover: 27 hours per service (soak) - SEQUENTIAL by environment
But can run 3-4 services in parallel across dev/staging/prod environments
Real time: ~7 days (with 3 parallel environments)

Total: Analysis (40 min) + Extraction (4 hours) + Manual (7 days) = ~7 days

Result: 56 days → 7 days (8x faster)

Failure Handling in Swarm

If Extractor Fails (One Service)

SCENARIO: Service 3 extraction times out

ACTION:

1. Extractor agent catches timeout exception
2. Sets: STATE.SERVICES[3].PHASE_3_STATUS = "failed"
3. Sets: STATE.SERVICES[3].PHASE_3_ERROR = "Timeout after 10 mins"
4. Sends email: "Service 3 extraction failed. Manual review needed."
5. Returns control to swarm

EFFECT:

- Service 3 is skipped (stays pending)
- Other services 4-34 continue normally
- Human can review, fix, and ask extractor to retry service 3
- Swarm continues without blocking

If Verifier Fails (One Service)

SCENARIO: Service 5 verification shows behavioral diffs

ACTION:

1. Verifier agent detects diffs
2. Sets: STATE.SERVICES[5].PHASE_4_STATUS = "failed"
3. Sets: STATE.SERVICES[5].PHASE_4_VERDICT = "FAIL"
4. Outputs: Detailed diff report
5. Sends email: "Service 5 verification failed. Extractor needs to fix."
6. Returns control to swarm

EFFECT:

- Service 5 is held in Phase 4 (doesn't proceed to Phase 5)
- Other services 1-34 continue normally
- Human can ask extractor to fix service 5 and re-verify
- Once fixed, verifier picks it up again automatically

If Hardener Fails (One Service)

SCENARIO: Service 8 is missing security control

ACTION:

1. Hardener agent detects missing TLS validation
2. Sets: STATE.SERVICES[8].PHASE_5_STATUS = "failed"
3. Sets: STATE.SERVICES[8].PHASE_5_ERROR = "Security control 2/6: TLS missing"
4. Sends email: "Service 8 hardening incomplete. Fix required."
5. Returns control to swarm

EFFECT:

- Service 8 stays in Phase 5 (doesn't get released for cutover)
- Other services 1-34 continue normally
- Human can ask hardener to add missing control
- Once fixed, service is marked ready for cutover

Monitoring the Swarm

Real-Time Dashboard

MONOLITH MIGRATION SWARM STATUS	

✓	PHASE 0: Analysis	[Complete] 10:10 am
✓	PHASE 1: Inventory	[Complete] 10:25 am
✓	PHASE 2: Domain	[Complete] 10:45 am
⏸	PHASE 3: Boundaries	[Awaiting Ratification]
EXTRACTION LOOP (Parallel Swarm):		
└ ✓	GetLoanAccountSupport	[Complete] 12:40
└ 🔄	AccountDetail	[Extracting] 12:05→ ...
└ 🔄	PaymentProcessing	[Verifying] 12:20→ ...
└ 🔄	OnlineWires	[Verifying] 12:25→ ...
└ ⌚	Audit	[Queue Position 5]
└ ⌚	Compliance	[Queue Position 6]
└ ⌚	Reporting	[Queue Position 7]
ACTIVE AGENTS:		
└	Extractor-1 [Service 2]	[10% complete]
└	Verifier-1 [Service 3]	[75% complete]
└	Verifier-2 [Service 4]	[20% complete]
└	Hardener-1 [Service 1]	[100% complete ✓]
└	Hardener-2 [Idle]	
PROGRESS:		
Completed: 1 / 34 services		
In Progress: 4 / 34 services		
Pending: 29 / 34 services		
ETA: 7 days 3 hours (based on current pace)		

Implementation: Swarm Coordinator Script

Create `.github/skills/monolith-decomposer/swarm-coordinator.py`:

```
#!/usr/bin/env python3
"""
Agentic Swarm Coordinator
Watches STATE.md, spawns agents, manages concurrency
"""

import json
import time
import subprocess
from pathlib import Path
```

```

from typing import Dict, List
from datetime import datetime

class SwarmCoordinator:
    def __init__(self, repo_root: str, config_file: str = None):
        self.repo_root = Path(repo_root)
        self.state_file = self.repo_root / "migration" / "STATE.md"
        self.config = {
            "max_parallel_extractors": 4,
            "max_parallel_verifiers": 6,
            "max_parallel_hardeners": 6,
            "timeout_extraction": 600,
            "timeout_verification": 900,
            "timeout_hardening": 900,
        }
        if config_file:
            self.load_config(config_file)

        self.active_agents = {} # {agent_id: process}

    def run_swarm(self):
        """Main loop: watch STATE.md and spawn agents"""
        while True:
            state = self.read_state()

            # Phase 0-2 (sequential)
            self.spawn_analyzer(state)
            self.spawn_inventory(state)
            self.spawn_domain(state)
            self.spawn_boundaries(state)

            # Phase 3-5 (parallel swarm)
            if state.get("PHASE_3_GATE", {}).get("STATUS") == "ratified":
                self.spawn_extractors_swarm(state)
                self.spawn_verifiers_swarm(state)
                self.spawn_hardeners_swarm(state)

            # Check for migration complete
            if self.is_migration_complete(state):
                print("🌈 SWARM MIGRATION COMPLETE")
                break

            time.sleep(2) # Check every 2 seconds

    def spawn_extractors_swarm(self, state: Dict):
        """Spawn up to MAX_PARALLEL_EXTRACTORS"""
        active_count = len([a for a in self.active_agents

```

```

        if "extractor" in a))

if active_count < self.config["max_parallel_extractors"]:
    # Find next service in pending state
    for service, data in state.get("SERVICES", {}).items():
        if (data.get("PHASE_3_STATUS") == "pending" and
            service not in self.active_agents):

            # Spawn extractor for this service
            agent_id = f"extractor-{service}"
            self.active_agents[agent_id] = {
                "service": service,
                "started": datetime.now(),
                "process": self.spawn_agent(
                    "migration-extractor",
                    service=service
                )
            }

def spawn_verifiers_swarm(self, state: Dict):
    """Spawn up to MAX_PARALLEL_VERIFIERS"""
    active_count = len([a for a in self.active_agents
                        if "verifier" in a])

    if active_count < self.config["max_parallel_verifiers"]:
        for service, data in state.get("SERVICES", {}).items():
            if (data.get("PHASE_3_STATUS") == "complete" and
                data.get("PHASE_4_STATUS") != "complete" and
                f"verifier-{service}" not in self.active_agents):

                agent_id = f"verifier-{service}"
                self.active_agents[agent_id] = {
                    "service": service,
                    "started": datetime.now(),
                    "process": self.spawn_agent(
                        "migration-verifier",
                        service=service
                    )
                }

def spawn_hardeners_swarm(self, state: Dict):
    """Spawn up to MAX_PARALLEL_HARDENERS"""
    active_count = len([a for a in self.active_agents
                        if "hardener" in a])

    if active_count < self.config["max_parallel_hardeners"]:
        for service, data in state.get("SERVICES", {}).items():

```

```

        if (data.get("PHASE_4_STATUS") == "complete" and
            data.get("PHASE_5_STATUS") != "complete" and
            f"hardener-{service}" not in self.active_agents):

            agent_id = f"hardener-{service}"
            self.active_agents[agent_id] = {
                "service": service,
                "started": datetime.now(),
                "process": self.spawn_agent(
                    "migration-hardener",
                    service=service
                )
            }

    def spawn_agent(self, agent_type: str, service: str = None):
        """Spawn a Copilot Agent Mode process"""
        # This calls Copilot Agent with the appropriate agent
        # Implementation depends on your Copilot integration
        pass

    def read_state(self) -> Dict:
        """Parse STATE.md YAML"""
        # Parse STATE.md and return dict
        pass

    def is_migration_complete(self, state: Dict) -> bool:
        """Check if all services are completed"""
        total = len(state.get("EXTRACTION_ORDER", []))
        completed = len(state.get("COMPLETED_SERVICES", []))
        return total > 0 and total == completed

if __name__ == '__main__':
    coordinator = SwarmCoordinator('.')
    coordinator.run_swarm()

```

Summary: Swarm vs Sequential

Aspect	Sequential	Swarm
Phases 0-2	40 minutes	40 minutes (parallel)
Extraction	1,300 hours (serial)	40 minutes (4 parallel)
Verification	1,700 hours (serial)	40 minutes (6 parallel)

Hardening	1,700 hours (serial)	40 minutes (6 parallel)
Manual Cutover	27 hours × 34 = 918 hours	~7 days (3 env parallel)
Total Time	56 days	~7 days
Complexity	Low (simple routing)	Medium (state coordination)
Failure Isolation	1 failure = stop all	1 failure = skip that service
Infrastructure	1 dev laptop	Need 3-4 parallel environments

Recommendation

Use Swarm if:

-  You have 3+ parallel environments (dev, staging, prod)
-  You want to migrate in days, not weeks
-  You can tolerate coordination complexity
-  You have 34 independent services

Use Sequential if:

-  You want simplicity and predictability
-  You have limited environments
-  You prefer single-machine operation
-  You want to debug/understand step-by-step

Start with Sequential, graduate to Swarm once you've done a few services and understand the flow.